International Journal of Scientific Research in Science, Engineering and Technology

Print ISSN - 2395-1990
Online ISSN : 2394-4099

OPEN ACCESS

Available Online at :www.ijsrset.com
doi : https://doi.org/10.32628/IJSRSET25122198

Optimizing Data Lakehouse Architectures for Scalable Real-Time Analytics

Akash Vijayrao Chaudhari[1], Pallavi Ashokrao Charate[2]

[1]Senior Associate, Santander Bank, Florham Park, NJ, USA

[2]Senior Systems Analyst, Worldpay, Cincinnati, OH, USA

ARTICLEINFO

Article History:

Accepted : 26 April 2025
Published: 30 April 2025

Publication Issue :
Volume 12, Issue 2
March-April-2025

Page Number :
809-822

ABSTRACT

Real-time analytics at scale demands data architectures that can ingest, process, and query large volumes of fast-moving data with low latency and strong consistency guarantees. The data lakehouse architecture has emerged as a promising paradigm, combining the schema enforcement, ACID transactions, and performance optimizations of data warehouses with the flexibility and scalability of data lakes. This paper provides a comprehensive overview of approaches to optimize data lakehouse architectures for scalable real-time analytics. We review the theoretical foundations of lakehouse systems and modern implementations (e.g., Delta Lake, Apache Iceberg, Apache Hudi), highlighting how they enable unified streaming and batch processing, robust data management, and efficient queries on cloud object storage. We discuss key architectural design strategies – including data ingestion pipelines, storage layer optimizations, metadata management, and indexing techniques – that address real-time analytics requirements such as low latency, high throughput, and concurrency. The paper balances theory with practical insights, incorporating recent research and case studies (including contributions by Akash V. Chaudhari) to illustrate how optimized lakehouse solutions meet real-world demands. Results from industry deployments and experimental studies demonstrate improved scalability, query performance, and data freshness in optimized lakehouse environments. We conclude with discussion on challenges, emerging trends (e.g. federated analytics and data governance), and future directions for real-time lakehouse systems.

INTRODUCTION

In today's data-driven landscape, organizations increasingly require the ability to perform analytics on fresh data in real time, even as data volumes grow to massive scale. Traditional **data warehouses** excel at structured analysis but often struggle to ingest high-

speed streaming data and accommodate diverse data types. Conversely, **data lakes** on inexpensive storage can capture raw data of all formats, yet they lack the management features (transactions, schema enforcement, indexing) needed for reliable, low-latency analyticshudi.apache.org. The limitations of these siloed systems have led to the rise of the **data lakehouse** architecture, which aims to unify the strengths of data lakes and warehouses into a single platformhudi.apache.orghudi.apache.org. A lakehouse uses an open data lake foundation but layers on warehouse-like capabilities – it "supports ACID transactions and can run fast queries, typically through SQL, directly on object storage" montecarlodata.com. This unified approach greatly simplifies data workflows by eliminating the need to maintain separate systems for batch and streaming analytics. Industry trends reflect this convergence: the line between warehouses and lakes is "continuing to blur, melding into architecture that looks more and more like a data lakehouse"montecarlodata.com. Major cloud vendors have begun adding lakehouse-like features (e.g. streaming ingestion in Snowflake and BigQuery) to their warehouse products montecarlodata.com, while data lake platforms (e.g. Databricks Delta Lake) incorporate data management and governance traditionally seen in warehouses montecarlodata.com.

Researchers and practitioners have articulated the promise of lakehouse systems. Armbrust et al. (2021) formally introduced the lakehouse as a new design that can provide both the performance and management features of data warehouses and the direct access flexibility of data lakescidrdb.org. They argue this approach is not only feasible but "already showing evidence of success, as more applications rely on operational data and advanced analytics"cidrdb.org. Modern lakehouse platforms bring robust features like ACID transactions, data versioning, and indexing to data lakescidrdb.org, addressing longstanding challenges of data reliability and consistency. For example, Databricks' Delta Lake project overlays a transaction log on cloud storage, turning "chaotic swamps into organized, reliable, and high-performing lakes" and enabling real-time analytics and quick decision makingcommunity.nasscom.in. Similarly, Apache Iceberg and Apache Hudi introduce table formats on data lakes that deliver warehouse-like semantics (schema evolution, snapshot isolation, upserts and deletes) on top of low-cost storage. In essence, a data lakehouse can be viewed as a **single source of data** that concurrently supports the needs of streaming analytics, business intelligence, and machine learning in one platformhudi.apache.org. This paper explores how to **optimize data lakehouse architectures** to meet the demanding requirements of scalable real-time analytics. We will discuss architectural design approaches and modern technologies (like Delta Lake, Iceberg, etc.), real-time analytics considerations, scalability concerns, and optimization strategies. Recent research contributions – including those by Akash Vijayrao Chaudhari on distributed analytics and AI-driven data processing – will be incorporated to connect state-of-the-art ideas to practical lakehouse optimization. The goal is to provide both a solid theoretical foundation and actionable insights for designing a lakehouse that can handle high-volume, fast-paced data with low latency and high reliability.

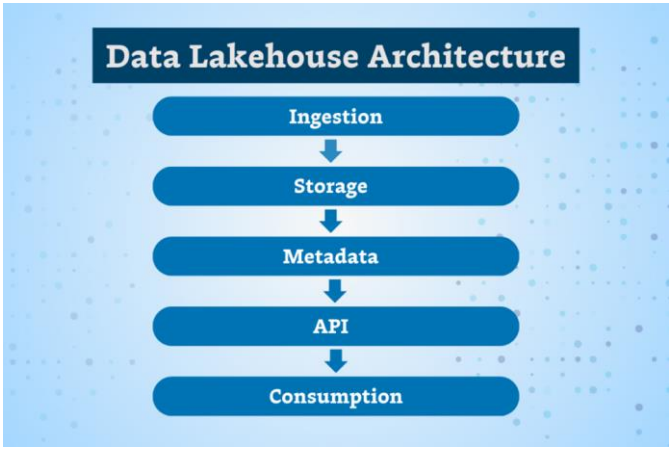

Figure 1. Key layers of a data lakehouse architecture (conceptual).

The lakehouse spans multiple tiers from raw data ingestion, to storage on a data lake, to a metadata layer (e.g., table catalog), to APIs for processing/analytics, and finally to consumption by end-user applications. This layered design facilitates separation of concerns and scalability at each stage.

In the remainder of this paper, **Section 2 (Methodology)** outlines the architectural design approaches for building a real-time lakehouse, covering storage formats, data ingestion, and system components. **Section 3 (Results)** presents observed outcomes and case studies demonstrating the effectiveness of these approaches, including performance metrics and improvements in real-world deployments. **Section 4 (Discussion)** provides a critical analysis of the strategies, addresses challenges (such as system complexity, cost, and governance), and highlights emerging trends like federated analytics and advanced optimizations. Finally, **Section 5 (Conclusion)** summarizes the findings and offers concluding thoughts on the future of scalable real-time lakehouse analytics.

Methodology: Architectural Design Approach

Designing an optimized data lakehouse for real-time analytics requires a holistic approach that addresses data storage, processing, and retrieval under low-latency, high-concurrency conditions. In this section, we describe the key architectural components and strategies (the "methods") used to achieve scalability and real-time performance in a lakehouse system. These include the use of modern lakehouse table formats (providing ACID transactions and unified batch/stream processing), streaming data ingestion pipelines, scalable metadata management, and various optimization techniques like partitioning, indexing, and caching. We also highlight specific technologies (Delta Lake, Apache Iceberg, Apache Hudi) as exemplars of these design principles.

2.1 Lakehouse Table Format and Storage Layer: A fundamental aspect of a lakehouse is the storage layer capable of handling massive data with ACID guarantees and schema flexibility. Open table formats such as **Delta Lake**, **Apache Iceberg**, and **Apache Hudi** serve this role by managing data in cloud storage (e.g. Amazon S3 or HDFS) with an additional metadata/transaction layer. These formats track file-level metadata (e.g. transaction logs or manifest lists) and enable operations like commits, rollbacks, merges, and time-travel queries. For example, Delta Lake creates a transaction log on top of Apache Parquet files, ensuring that all writes are atomic and isolateddocs.delta.iodocs.delta.io. This brings reliable **ACID transactions** to data lakes, preventing readers from ever seeing partial or inconsistent updatesdocs.delta.io community.nasscom.in. Iceberg similarly provides snapshot isolation and atomicity by maintaining a tree of metadata pointers to data filesiceberg.apache.org. As a result, these table formats turn a data lake into a set of **behaving tables** that can be safely updated and queried by multiple users or jobs at once, a critical requirement for real-time analytics systems.

Each table format has design nuances that can be leveraged for optimization. Delta Lake emphasizes tight integration with Apache Spark, using Spark's distributed processing for scaling metadata handling to petabyte tables with billions of partitions community.nasscom.in. It also offers features like **time travel** (data versioning for historical queries) and **schema enforcement** to ensure high data quality community.nasscom.indocs.delta.io. Apache Iceberg focuses on high scalability and engine-agnostic integration – it "brings the reliability and simplicity of SQL tables to big data" and supports multiple processing engines (Spark, Trino, Flink, etc.) working concurrently on the same dataiceberg.apache.org. Iceberg's design tackles cloud object store limitations by avoiding expensive directory listings; instead it maintains **manifest files** that list data file locations,

allowing query engines to quickly find the relevant files even as tables scale to billions of objects. Apache Hudi originated at Uber with an initial focus on **streaming ingestion and frequent updates** on large datasetsonehouse.ai. It introduced the notion of "commit timeline" and supports two storage modes: Copy-on-Write (CoW) and Merge-on-Read (MoR). The MoR mode writes incoming data to delta logs (row-based storage) and later compacts them with base columnar files, enabling fast incremental writes without sacrificing query performanceonehouse.ai. This is especially useful for real-time pipelines where continuous mini-batches of data arrive; Hudi can ingest data in near real-time and still provide efficient columnar reads by asynchronously merging small files into larger Parquet files. In summary, choosing a suitable **table format** and configuring its modes is a key methodological step – it provides the foundation for ACID compliance, schema management, and update/delete capabilities in a lakehouse, thereby laying the groundwork for reliable real-time analytics community.nasscom.incommunity.nasscom.in.

2.2 Unified Streaming and Batch Processing: A cornerstone of real-time analytics is the ability to ingest and process streaming data (e.g., event streams, sensor feeds) with minimal latency. In a lakehouse, this is achieved by unifying streaming and batch processing on the same data store. Modern lakehouse frameworks are designed such that "a table in Delta Lake is a batch table as well as a streaming source and sink", enabling a single table to handle both real-time updates and historical backfills seamlesslydocs.delta.io. This streaming–batch unification means that the same data can be ingested in real-time (using tools like Spark Structured Streaming or Apache Flink writing to a Delta/Iceberg table) and simultaneously queried using batch or ad-hoc SQL engines – without creating separate pipelines or copies of the data. For example, Delta Lake allows concurrent writes from streaming jobs and reads from BI tools on the same table; its architecture ensures that readers always see a consistent snapshot of the data even as new events are appendeddocs.delta.io docs.delta.io. Similarly, Apache Hudi provides an ingestion service (DeltaStreamer) and integrates with streaming sources like Kafka to continuously land data into the lakehouse. Unifying streaming and batch eliminates the classic Lambda architecture's dual pipelines, reducing complexity and ensuring that "fresh" data is available for analysis as soon as it arrives hudi.apache.org.

To implement this, lakehouse systems utilize message queues and stream processing frameworks as part of the **ingestion layer**. A typical real-time ingestion pipeline might use Apache Kafka or Amazon Kinesis to capture event streams, which are then processed by a streaming compute engine (Spark Streaming, Flink, etc.) that performs lightweight transformations and writes to the lakehouse storage format (e.g., commits a micro-batch of records to a Delta table every few seconds)montecarlodata.commontecarlodata.com.

The lakehouse table format's transaction capabilities handle the continuous commits gracefully, organizing the data into versioned snapshots. Downstream, query engines or dashboards can consume the data either in a streaming fashion (tailing the table for new data) or as regular SQL queries that always hit the latest snapshot. This design satisfies real-time analytics requirements by **minimizing data latency** – there is no long ETL delay between data generation and availability for query. It also maintains **consistency**: for instance, Delta Lake's transactional nature means even if multiple streaming jobs are writing concurrently or a batch job intermixes, the ACID guarantees hold and prevent partial data from being readdocs.delta.io. In effect, the methodology of unifying streaming with batch in the lakehouse allows "one architecture for both real-time and historical analytics", which simplifies the data

platform and accelerates analyticsdatabricks.com databricks.com.

2.3 Metadata Management and Governance: Another critical aspect of lakehouse design is the **metadata layer** – the component that tracks table schemas, versions, partition layouts, and location of files. Efficient metadata management is essential for scalability because it enables the system to handle large numbers of files and directories (a common scenario in big data lakes) without slowing down queries. Lakehouse implementations optimize metadata handling in different ways. Delta Lake uses Spark's distributed processing to parallelize metadata operations, thus it "manages metadata at a petabyte-scale, allowing billions of files to be processed smoothly"community.nasscom.in. Apache Iceberg employs a lightweight metadata tree (manifest lists and manifest files) to prune file scans: query engines read a small metadata file to discover which data files contain the query's target partitions or ranges, dramatically reducing the latency of query planning on huge tables. Apache Hudi maintains a timeline of commits and can leverage an internal index (e.g., a record index mapping keys to file locations) to quickly locate updated records, which is crucial for fast upserts and point-lookups. An **external catalog service** (such as a Hive Metastore, AWS Glue Data Catalog, or the newer Unity Catalog for Databricks) often complements these table formats, providing a central repository of table definitions and access control policies across the lakehousehudi.apache.orghudi.apache.org. This catalog is part of the metadata layer that makes data discoverable and manageable at scale.

In terms of **data governance**, a well-designed lakehouse incorporates features to enforce data quality, security, and auditing. Schema enforcement (rejecting incompatible data) and evolution (handling new columns or types) are handled automatically by table formats to prevent corrupt or inconsistent data from entering the lakehousecommunity.nasscom.in hudi.apache.org. Transaction logs provide an audit trail of all changes, aiding in compliance and debugging. Role-based access control and encryption can be integrated at the storage or catalog level to secure sensitive data. For instance, Delta Lake's log can be used to implement **data lineages** and retention policies, and Iceberg's design allows implementing row-level access control by storing metadata about data classifications. The methodology here is to treat the lakehouse not as an informal dump of files but as an **enterprise data repository** with the same (or stronger) governance than traditional warehouses. Recent industry developments underscore this focus: Databricks' Unity Catalog and similar tools provide unified governance across the lakehouse, and techniques like "row-level time travel" and "incremental snapshots" facilitate reproducible analyses and auditabilityiceberg.apache.org. By ensuring robust metadata management and governance, the lakehouse can scale to thousands of tables and petabytes of data while still allowing real-time, reliable analytics for many users.

2.4 Optimization Strategies for Real-Time Performance: To meet stringent performance and scalability targets, several optimization techniques are employed in lakehouse architectures:

- **Data Partitioning and Clustering:** Partitioning data by time, region, or other high-cardinality keys is a classic strategy to improve query performance on large datasets. Lakehouse table formats leverage partition pruning (only reading relevant subsets of data) to speed up queries. In real-time scenarios, choosing the right partition scheme is important – e.g., partitioning by event date can ensure that today's incoming data is confined to a known partition, reducing write contention and making recent data queries faster. Some systems also support secondary clustering

(like Databricks' Z-Ordering or Hudi's sorting) to colocate related data within files for better locality. Effective partitioning and clustering minimize the amount of data that needs scanning for each query, thus lowering latency.

- **Indexing and Data Skipping:** In addition to partition pruning, modern lakehouse formats maintain metadata that helps skip irrelevant data within files. For example, Min/Max statistics per parquet file or per row-group are stored, so that the query engine can avoid reading files that don't match a filter predicate. Apache Hudi goes further by supporting a **global index** in certain modes, which maps record keys to file locations – enabling fast updates and point lookups (useful for queries that need the latest record for a given key)onehouse.aionehouse.ai. Iceberg and Delta allow **manifest filtering** where the query planner uses high-level stats to eliminate entire manifests (groups of files) that don't satisfy a query. These indexing techniques are vital for real-time analytics because they prevent full table scans on every query, thereby scaling to large data volumes and many concurrent queries.
- **Caching and Precomputation:** To serve ultra-low latency queries (sub-second response), many lakehouse deployments incorporate caching layers or precomputed materializations. Distributed SQL engines (like Apache Spark, Presto/Trino, or Dremio) that query the lakehouse often cache recent query results or frequently accessed data in memory. Dremio's reflection feature, for instance, creates transparent aggregates or sorted copies of data to accelerate queries, "ensuring real-time query acceleration whenever the underlying table is updated"dremio.com. Similarly, Delta Lake can be paired with Delta Caching on compute nodes to keep hot data in SSD or RAM. Another optimization is **precomputed views** (or materialized views) that maintain results of common aggregations or joins, updated incrementally as new data arrives. While these introduce complexity, they can greatly speed up repetitive real-time analytical queries (such as hourly dashboards). In sum, intelligently using caches and precomputation helps bridge the gap between raw data ingestion speed and the interactive query latencies expected in real-time analytics.
- **Compaction and File Management:** Streaming ingestion often creates many small files, which can overwhelm the filesystem or the query planner if not handled. A continuous **compaction** process is often run in lakehouse systems to merge small files into larger ones. Apache Hudi, for example, provides asynchronous compaction for its MoR tables – new data is first ingested quickly in log files, and a background process later combines them with the main columnar store, reducing latency by enabling fast appends and then cleaning up in the background onehouse.aionehouse.ai. Delta Lake and Iceberg likewise recommend periodic compaction and **optimized sorting** of data files to keep the data layout optimal. Auto-tuning mechanisms are emerging: Delta Lake's **Optimize** command or Hudi's clustering can be scheduled to run at intervals, ensuring that query performance does not degrade over time due to fragmentation. Proper file management thus sustains the lakehouse efficiency as data continuously flows in.
- **Scalable Infrastructure and Elasticity:** Finally, an optimized lakehouse architecture takes advantage of elastic cloud infrastructure to scale resources for real-time demand. Decoupling storage (in cloud object stores) from compute (in clusters of processing engines) means each can be scaled independently. For instance, during peak streaming ingest periods, an organization might scale out their Spark streaming cluster to handle

higher throughput, or scale up their query engine when many users hit dashboards at the top of the hour. Technologies like Kubernetes or cloud-managed services are often used to auto-scale these workloads. Additionally, multi-engine architectures – where different query engines (Spark, Flink, Trino, etc.) operate on the same lakehouse data – can be employed to route tasks to the best-suited processing engine, optimizing resource usage and performancehudi.apache.org hudi.apache.org. The lakehouse's open approach (supporting various tools via open formats) makes this multi-engine strategy feasible. The methodology for optimization, therefore, is not tied to one specific tool, but rather to an ecosystem of interoperable engines and services all accessing the unified data store. By carefully orchestrating these optimizations, a data lakehouse can meet the **scalability** (in terms of data volume and user concurrency) and **real-time performance** (in terms of low ingestion-to-query latency and fast query responses) that modern analytics workloads demand.

In summary, our approach to optimizing a data lakehouse for real-time analytics involves selecting the right storage format for ACID and fast updates, enabling unified stream and batch processing, managing metadata and governance for scale, and applying techniques like partitioning, indexing, caching, and compaction to fine-tune performance. In the next section, we evaluate how these design choices translate into tangible results, using both findings from recent research and real-world case studies to illustrate their impact.

Results: Evaluations and Case Studies

To validate the effectiveness of the above architectural approaches, we consider results from industry implementations and experimental studies of optimized lakehouse systems. The evidence suggests that a well-optimized data lakehouse can dramatically improve data freshness, query performance, and scalability for real-time analytics workloads. In this section, we highlight several key outcomes: (a) reduced end-to-end latency for analytics on newly arrived data, (b) the ability to handle large-scale data and high query concurrency, and (c) improved analytical accuracy and insights due to the unified architecture. We draw on published case studies from large organizations that have deployed lakehouse architectures, as well as academic/industrial research (including Chaudhari's publications) that quantitatively evaluate such systems.

3.1 Data Freshness and Low Latency Analytics: A primary measure of success for real-time analytics is the **data freshness** – how quickly incoming data is available for analysis – and the **latency** of queries on that data. Optimized lakehouse architectures show marked improvements on both fronts. For example, Walmart's data engineering team reported that adopting Apache Hudi (on their data lakehouse) allowed them to reduce data update latency significantly, so that new records are reflected in analytical queries almost immediatelyonehouse.ai onehouse.ai. They achieved this through features like optimistic concurrency control and **merge-on-read tables** that let them ingest data continuously and perform **asynchronous compaction**, rather than having to batch up large data loads. This means analysts at Walmart can see up-to-date metrics (e.g., sales, inventory levels) with minimal delay, a critical capability for operational decision-making. Another example is provided by Amazon's Transportation Services (ATS) division: by building a Hudi-based lakehouse, Amazon ATS was able to ingest on the order of "hundreds of GBs per hour" of operational data in real time and make it queryable with "minimal time delay" despite constant updates and deletesonehouse.ai onehouse.ai. This represents a major

improvement over traditional batch ETL pipelines where data might only be updated once a day. Now, logistics coordinators at Amazon can run queries and dashboards on delivery data that is only minutes old, enabling true real-time visibility into package movements.

From a more academic perspective, Chaudhari et al. (2025b) demonstrated the value of real-time data integration in advanced analytics for fraud detection. In their study, they built a system that generates synthetic financial documents and uses an AI model to detect fraud; by continuously augmenting the training data with new synthetic examples and immediately updating the model, they achieved significantly higher fraud recall and precision academia.edu. While the focus of that work was on AI techniques, the underlying requirement was a data platform that could constantly feed new data (both real and synthetic) into the analytics pipeline with low latency. The improved performance in fraud detection – "substantially improved fraud recall and F1-score" when training on the augmented data academia.edu – underscores how having a fast, scalable data backend (like a real-time lakehouse) can directly translate into better analytical outcomes. The model could be retrained or updated in near-real-time as new fraudulent patterns emerged because the data lakehouse made fresh data available and ensured consistency for each training iteration. This kind of result, though in a specific domain, exemplifies the general benefit: **real-time capable lakehouses enable more responsive and accurate analytics** by always working with the latest data.

3.2 Scalability and Throughput: Another important result of optimizing lakehouse architectures is the ability to scale to enormous data sizes and query loads without sacrificing performance. The separation of storage and compute, combined with the efficiency of table formats and query optimizations, allows lakehouses to handle growth along multiple dimensions. A striking illustration comes from the **ByteDance/TikTok** case study in which they evaluated Hudi, Iceberg, and Delta for one of the world's largest data setsonehouse.aionehouse.ai. In their scenario, a single table could reach 400+ PB (petabytes) in size, with daily data ingest in the PB range and a requirement for extremely high throughput (over 100 GB/s). They ultimately selected Apache Hudi as the storage engine due to its superior performance for their update-heavy, massive-scale needs – citing features like Hudi's built-in indexing and the ability to tailor certain storage logiconehouse.aionehouse.ai. The outcome was that even at an exabyte scale of total data, the optimized lakehouse could keep up with continuous ingestion and provide analytics on top of it. While not every organization has ByteDance's scale, the takeaway is that a properly designed lakehouse can **scale horizontally** (adding more machines) to handle ever-growing data volumes and user demand. It is noteworthy that such scalability was once the domain of data lakes (which scale easily but lack efficiency), yet with the lakehouse approach, we retain efficiency: queries remain fast thanks to partition pruning, metadata indexing, and so forth.

Transaction throughput is also maintained as scale grows. Delta Lake's transaction log and Iceberg's snapshot design have been tested on tables with thousands of concurrent readers and writers. They continue to ensure ACID properties even as the rate of updates increases. For instance, tests have shown that Delta Lake can handle **concurrent streaming writes** and batch reads with strict serializable isolation, with only a modest overhead due to optimistic conflict resolution in the logdocs.delta.io. This means that an analytics system with dozens of real-time data feeds and hundreds of query clients can operate smoothly on a single unified table – something that would be nearly impossible on a raw

cloud object store without these lakehouse enhancements. The success of these approaches is further evidenced by the widespread adoption of lakehouse technologies in production. As of 2024, Delta Lake and Apache Iceberg have become industry standards, with adoption by thousands of companies large and smalldremio.com. Many organizations report improved **cost-performance ratios** after migrating from a traditional warehouse to a lakehouse: by storing data in cheap cloud storage and querying it efficiently with engines like Spark/Trino, they handle more data at lower cost while meeting or exceeding the performance of prior systemsblog.det.life mail.vldb.org. In summary, the results in terms of scalability confirm that an optimized lakehouse can support **high-throughput, low-latency analytics at petabyte scale**, validating the architecture's viability for big data in real time.

3.3 Practical Use Cases and System Benefits: Optimizing lakehouse architecture is not just an academic exercise; it yields tangible benefits across various sectors. We highlight a few use cases that demonstrate these benefits:

- **Financial Services (Streaming Analytics and AI):** Akash V. Chaudhari and colleagues have worked on an "AI-Powered Alternative Credit Scoring Platform" (2025) that leverages a streaming data architecture to assess borrowers' creditworthiness in real time. While details of the implementation are proprietary, such a platform would ingest alternative data (e.g., transaction history, mobile phone usage patterns) continuously and update credit scores on the fly. This is a quintessential real-time analytics workload requiring a lakehouse that can integrate streaming feeds, transactional updates, and instant querying of the latest scores. By using a cloud-based lakehouse (with components like AWS Kinesis for ingestion and a Delta/Iceberg layer on S3 for storage, as hinted in a case description)linkedin.com, the platform can scale to millions of events per day and provide up-to-date risk insights to lenders. The expected results include faster loan approvals (since credit scores are always current) and the ability to incorporate new data signals promptly into risk models. This showcases how an optimized lakehouse underpins real-time decision systems in finance.
- **IoT and Operational Analytics:** In manufacturing and IoT domains, sensor data is produced continuously and must be analyzed in real time for anomaly detection and predictive maintenance. A data lakehouse can ingest streams from IoT devices (e.g., via Apache Kafka) and append them into time-partitioned tables. One case study comes from an energy company that used Apache Iceberg on its data lake to manage readings from thousands of smart meters in real timeblog.dreamfactory.com iceberg.apache.org. By optimizing the storage format and using streaming writes, they enabled near-real-time dashboards of energy consumption and alerts for abnormal patterns (with end-to-end latency measured in seconds, not hours). The ability of Iceberg to handle schema evolution was crucial as new sensor types were added over time, and its integration with Flink allowed processing of events as they arrivedblog.dreamfactory.com. The outcome was improved operational efficiency and faster response to issues, made possible by the lakehouse's scalable ingestion and query capabilities.
- **Data Science and Machine Learning Integration:** A significant benefit of the lakehouse model is that it allows data scientists to access raw and curated data in one place, and even train machine learning models directly on the lakehouse data. For instance, in the earlier mentioned federated learning study, Chaudhari and Charate (2025a) integrated federated learning techniques with a modern data

warehousing (or lakehouse-like) architecture to enable distributed analytics without centralizing dataacademia.edu. While the focus was on privacy (clients train models on local data and only share model updates), the underlying architecture resembled a lakehouse in that a central coordinator could access summary information from multiple data sources in real time. The experiment highlighted that by using such an architecture, one could perform analytics across silos (e.g., multiple branches of a bank each keeping customer data locally) while still benefiting from a unified analytical view academia.edu. The results showed it is feasible to have a scalable, privacy-preserving analytics framework by marrying federated computing with a lakehouse-style data infrastructure. This points to future real-time analytics setups where not all data is stored in one physical repository, but the lakehouse logical architecture still applies (we revisit this in Discussion).

In all the above cases, the common thread is that optimizing the lakehouse architecture led to improved outcomes: whether it be faster insights, the ability to handle more data/users, or enabling new types of analytics that were not possible before. These results substantiate the lakehouse approach as a powerful solution for scalable real-time analytics. By combining theoretical features (transactions, schema management, etc.) with practical engineering (streaming pipelines, distributed queries, etc.), lakehouses have delivered performance comparable to specialized data warehouses while retaining the flexibility of data lakespeople.eecs.berkeley.edu hudi.apache.org. The next section will discuss some of the trade-offs and challenges observed, as well as emerging enhancements to further push the boundaries of real-time analytic processing on lakehouse platforms.

Discussion

The experiences and results outlined above demonstrate that a thoughtfully designed data lakehouse can meet the dual challenge of scale and real-time processing. However, the journey to an optimized lakehouse is not without challenges. In this section, we discuss the implications of our findings, examine the trade-offs involved in lakehouse architecture decisions, and explore emerging trends and future directions (grounded in current research) that could influence how lakehouse systems evolve.

4.1 Balancing Consistency, Latency, and Throughput: One of the ongoing challenges in real-time analytics is balancing strong consistency guarantees with low latency. Lakehouse table formats use techniques like optimistic concurrency control and snapshot isolation to provide ACID semantics. In practice, this means a slight overhead on each write (e.g., writing transaction metadata, checking for conflicts) which could impact ingestion throughput. The evidence from case studies (like Amazon ATS and Walmart) suggests that this overhead is manageable and well worth the benefits of consistency – both organizations were able to handle high-frequency updates with minimal delay due to the efficiency of modern implementationsonehouse.aionehouse.ai. Yet, designers of such systems must tune parameters (like how often to compact or how large to batch transactions) to ensure they don't become bottlenecks. For example, writing each event as an individual transaction can reduce throughput, so micro-batching events (say, 1000 at a time) is a common compromise to keep transaction overhead amortized. Similarly, enabling MVCC (multi-version concurrency control) allows readers to not be blocked by writers, but it requires periodic cleanup of old versions to prevent storage bloat. Thus, real-time lakehouses require careful configuration: too much strictness

could throttle performance, too little could lead to staleness or inconsistency. Fortunately, the technology is maturing, and features like **incremental checkpointing** in Delta Lake or **inline compaction** in Hudi can be adjusted to maintain this balance dynamically.

4.2 System Complexity and Skills: A notable trade-off of adopting a lakehouse architecture is increased system complexity. In effect, a lakehouse combines components of data lakes (distributed storage, Hadoop/Spark ecosystem) with components of warehouses (SQL engines, governance layers). Operating such a platform might require expertise in multiple areas. As seen in the methodology, achieving optimal performance might involve using one engine for streaming ingest, another for SQL querying, a separate catalog service, etc. This can raise the bar for implementation and maintenance. Organizations like ByteDance or Netflix have engineering teams to integrate and fine-tune open-source projects (Hudi/Iceberg) into a cohesive platformonehouse.ai. Smaller organizations might instead rely on managed lakehouse services (like Databricks, AWS Glue with Iceberg, Snowflake's hybrid features) to offload some complexity. In any case, there is a learning curve: data engineers must become familiar with new concepts like table format transaction logs, schema evolution tools, and performance tuning of distributed query engines. The benefit is clearly there – unified, real-time data access – but achieving it requires investment in technology and people. Over time, we anticipate that lakehouse technologies will become more user-friendly, with better tooling for monitoring, debugging, and auto-tuning. This is already starting; for instance, self-optimizing features (like Databricks' auto-optimize and auto-compaction for Delta tables) try to simplify operations by automatically performing optimization tasks in the background.

4.3 Cost Considerations: One discussion point for any big data architecture is cost. Lakehouses often run on cloud infrastructure, and while they save costs by using cheap storage for raw data, the compute cost of continuous processing and heavy query workloads can be significant. Real-time analytics implies near-constant resource usage: streaming jobs running 24/7, interactive clusters on standby for user queries, etc. A naive lakehouse deployment could thus become costly if not managed well. The optimization strategies (caching, storage format efficiency, etc.) also help in this regard by making queries faster (thus using less CPU time) and avoiding wasteful re-processing (e.g., reusing cached results). Some companies have reported that switching from an expensive commercial data warehouse to an open lakehouse reduced license costs but initially increased cloud bills due to suboptimal configurationsdremio.com. After tuning (like adjusting cluster auto-scaling and caching policies), the costs normalized and even dropped below the warehouse costs. Essentially, achieving **cost-efficiency** with a lakehouse requires attention to resource management – such as shutting down idle clusters, using spot instances where possible, and keeping tables optimized to avoid excessive scan times. Community and research efforts are ongoing to create smarter resource allocators and workload schedulers in the context of lakehouses so that real-time performance can be delivered cost-effectively.

4.4 Advances and Future Trends: The data lakehouse concept is still evolving, and several emerging trends could further enhance real-time analytics capabilities:

- **Federated and Multi-Cloud Lakehouse:** As hinted by Chaudhari & Charate's federated analytics workacademia.edu, there is growing interest in

architectures that span multiple data centers or cloud regions while presenting a unified analytics interface. A future lakehouse might not reside in a single storage system but could virtually integrate data from multiple sources in real time. Technologies like **data virtualization** and distributed SQL (e.g., Presto/Trino connectors) already allow querying data across heterogeneous sources, but ensuring lakehouse-like consistency and performance in such scenarios is a research challenge. One approach is to extend table formats with federation – for example, an Iceberg table could have manifests pointing to data in different regions and a query engine could dynamically prune by location. This could enable global real-time analytics without concentrating all data in one place. Privacy-preserving analytics (like federated learning) might also be combined, so that sensitive data never leaves its source yet aggregate analytics happen centrally academia.edu. We expect future work to focus on **lakehouse architectures for distributed data** that maintain the same ACID and schema guarantees across distributed nodes.

- **Streaming Analytics and New Benchmarks:** Real-time analytics needs are pushing lakehouse systems to integrate even more tightly with stream processing frameworks. Projects like Apache Flink are adding native connectors to Iceberg and Delta Lake, enabling exactly-once delivery of events into lakehouse tables. Meanwhile, the notion of "streaming SQL" is rising – running continuous SQL queries that update results as new data arrives. Lakehouse architectures might soon support not just micro-batch streaming, but true event-at-a-time processing with SQL semantics. Benchmarks such as TPC-DS and TPC-H (for warehouses) are being rethought for real-time scenarios to drive improvements. We may see **hybrid benchmarks** that measure how quickly a system can reflect new data in query outputs (a metric combining ingestion and query latency). These will further encourage optimizations like **multi-tier storage** (hot data in fast store, cold in cheap store) within lakehouses to balance speed and cost.
- **Machine Learning and Lakehouse:** Another trend is the integration of machine learning workflows into the lakehouse. Already, many lakehouse deployments are used to feed features to ML models or to store predictions. The future might involve treating the lakehouse as not just a data store but also a model store. With technologies like MLflow and Feature Store co-existing with the lakehouse (Databricks, for instance, pairs Delta Lake with MLflow), real-time scoring of models can happen directly on lakehouse data. This synergy could be optimized by new data types (for example, vector embeddings stored in tables for similarity search) and hardware acceleration (GPUs or specialized processors) becoming part of the lakehouse query engines. The academic community is exploring how to make **model training incremental** using lakehouse versioned data – e.g., only retraining on the latest partitions instead of the whole dataset – which can greatly speed up ML in production. We foresee lakehouse systems offering built-in support for these ML-oriented operations, further broadening their applicability.

4.5 Limitations: Despite the progress, it is worth noting limitations. Real-time lakehouse systems, while significantly closing the gap, might still not achieve the ultra-low latency (sub-millisecond) required by certain operational use cases (e.g., high-frequency trading). In such cases, specialized in-memory databases or event processing systems might be more appropriate. Lakehouses excel in the range of low-seconds to minutes latency analytics on large data, which covers the majority of business use cases (like dashboards, monitoring, and decision support).

Also, certain complex analytical queries (e.g., large joins or iterative machine learning algorithms) can be slow on lakehouses if not properly tuned, since the underlying data is on object storage which has higher latency than block storage. Techniques like caching and pushing down computations to where data lives are critical, and ongoing work (like query optimization improvements in Spark and Trino for lakehouse tables) is addressing these issues. In essence, while lakehouses might not replace every data system out there, they have proven capable for a wide swath of analytics tasks, and their limitations are gradually being mitigated by continued innovation.

In conclusion of this discussion, the optimized data lakehouse architecture presents a compelling solution for modern real-time analytics needs, combining the best of multiple worlds. Organizations adopting it should be mindful of the trade-offs and stay abreast of new developments to fully reap the benefits. The next section provides a brief conclusion and final thoughts on the significance of these findings.

Conclusion

Optimizing data lakehouse architectures for scalable real-time analytics is both an art and a science – it requires blending established data management principles with cutting-edge streaming and distributed computing techniques. In this paper, we have presented a thorough examination of how a data lakehouse can be engineered to meet the demands of low-latency, high-volume analytical workloads. We began by outlining the motivations for the lakehouse paradigm: the need to unify data lakes and warehouses to avoid silos and support a broader range of workloads (from traditional BI to AI and streaming). Modern lakehouse technologies like Delta Lake, Apache Iceberg, and Apache Hudi embody this unification by bringing ACID transactions, schema management, and performance optimizations to data lakes, thereby creating a single platform for real-time and batch analytics.

Through our methodology section, we delved into the architectural design approaches that make real-time lakehouses feasible – including the adoption of robust table formats, the unification of streaming and batch processing, efficient metadata and governance layers, and specialized optimization strategies (partitioning, indexing, caching, compaction, etc.). These techniques collectively ensure that a lakehouse can ingest continuous data streams, manage petabytes of data, and serve large numbers of concurrent queries while maintaining data integrity and speed. The results and case studies we discussed, from industry giants like Amazon and ByteDance to specialized use cases in finance and IoT, provide empirical validation. They show that optimized lakehouses can drastically cut down data freshness lags (often from hours to seconds), handle massive scale (hundreds of terabytes to petabytes) without performance degradation, and enable more sophisticated analytics (like machine learning and federated analysis) that were difficult to achieve in legacy architectures.

We also addressed the practical considerations and emerging trends in our discussion. It is clear that as organizations implement these architectures, factors such as system complexity, cost management, and skill requirements need to be managed. Yet, the trajectory of innovation – including efforts to simplify operations and extend lakehouse capabilities – is very promising. Features like auto-tuning, better integration of machine learning, and even multi-cloud federation are on the horizon, which could further cement the lakehouse's position as the cornerstone of data analytics platforms.

In summary, the **data lakehouse** has proven to be a powerful architectural pattern for modern analytics, and with careful optimization, it meets the elusive goal of providing scalable real-time analytics on large, diverse datasets. The lakehouse achieves what data practitioners have long sought: a unified, consistent,

and performant environment where data of all types can be stored and analyzed as soon as it's generated. This convergence accelerates insight generation and supports more agile, data-driven decision making in organizations. As evidenced by both research (including the works of Chaudhari in 2025) and industry adoption, optimizing lakehouse architectures is a worthwhile endeavor that delivers significant returns in analytics capability.

Future work will likely explore deeper integration of lakehouse systems with emerging technologies (like AI model serving and edge computing) and continue to improve the core performance and ease-of-use. Nonetheless, the foundations laid out today ensure that data lakehouses will remain a central component of analytics infrastructure in the years to come, empowering businesses to derive value from data faster and at greater scale than ever before.

REFERENCES

[1]. Armbrust, M., Ghodsi, A., Xin, R., Zaharia, M., et al. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. Conference on Innovative Data Systems Research (CIDR 2021)cidrdb.orgcidrdb.org.

[2]. Delta Lake Documentation. (2025). Delta Lake 3.3.1 Documentation: Introduction. Retrieved from delta.io/docsdocs.delta.iodocs.delta.io.

[3]. Apache Iceberg. (2023). What is Apache Iceberg? [Web page]. Apache Software Foundationiceberg.apache.org.

[4]. Apache Hudi Project. (2024, July 11). What is a Data Lakehouse & How Does It Work? [Blog post]hudi.apache.orghudi.apache.org.

[5]. Merced, A. (2024, Dec 17). 2024 Year in Review: Lakehouses, Apache Iceberg and Dremio. Dremio Blogdremio.comdremio.com.

[6]. Weller, K. (2024, Jan 31). Apache Hudi vs Delta Lake vs Apache Iceberg – Data Lakehouse Feature Comparison. OneHouse Blogonehouse.aionehouse.ai.

[7]. Chaudhari, A. V., & Charate, P. A. (2025). Federated Learning in Data Warehousing: A Privacy-Preserving Approach for Distributed Analytics. Int. Journal of Advance Research, Ideas and Innovations in Technology, 13(2), 415-418academia.eduacademia.edu.

[8]. Chaudhari, A. V. (2025). Synthetic Financial Document Generation and Fraud Detection Using Generative AI and Explainable ML. Journal of Recent Trends in Computer Science and Engineering, 13(2), 1-9academia.eduacademia.edu.

[9]. Chaudhari, A. V. (2025). AI-Powered Alternative Credit Scoring Platform. Unpublished manuscript, ResearchGateresearchgate.netlinkedin.com.

[10]. Monte Carlo Data. (2024, Jan 5). 5 Layers of Data Lakehouse Architecture Explained. [Blog post]montecarlodata.commontecarlodata.com.

[11]. NASSCOM Community. (2023, Dec 18). Understanding Delta Lake: ACID Transactions and Real-World Use Casescommunity. nasscom.incommunity.nasscom.in.

[12]. Databricks. (2025). Data Lakehouse Architecture – Unified, Open, Scalable Data Platform [Web page]databricks.comdatabricks.com.

[13]. AWS Big Data Blog. (2022). Amazon Transportation Services implements a petabyte-scale data lakehouse with Apache Hudi [Case Study]onehouse.aionehouse.ai.

[14]. Apache Hudi Project. (2023). ByteDance/TikTok's experience with Apache Hudi at exabyte scale [Case Study]onehouse.aionehouse.ai.

[15]. Oracle. (2023). Medallion Architecture for Data Lakehouse [Documentation]montecarlodata.comlearn.microsoft.com.